Backend de Dietética

Guía sencilla para conectar React con Node, Express, Sequelize y SQLite

Basada en el patrón del proyecto del profesor · La API ya está preparada en la carpeta Backend

> **La idea en una línea:**
>
> **React (useEffect) → fetch → Express (ruta) → servicio → Sequelize (modelo) → dietetica.db**
>
> React no abre el archivo de base de datos. Pide los datos por HTTP; el servidor Node consulta SQLite por medio de Sequelize y responde en JSON.

Paso 1. Qué instalamos (ya está configurado)

El backend usa las mismas tecnologías principales del ejemplo del profesor:

Tecnología	Para qué sirve
Node.js	Ejecuta JavaScript en el servidor.
Express	Recibe rutas como GET /api/productos.
Sequelize	ORM: convierte operaciones sobre modelos en consultas SQL.
SQLite / sqlite3	Almacena la información en dietetica.db.
morgan	Muestra en la terminal las solicitudes que llegan a la API.

En la terminal, situada en la carpeta del backend:

```
npm install
npm run dev
```

El servidor escucha en `http://localhost:3000`. Para detenerlo, presioná `Ctrl+C`.

Paso 2. Entender las carpetas (igual que el patrón del profesor)

```
routes/                  URLs y métodos HTTP
  products.routes.js
  users.routes.js
src/
  index.js               Inicia Express y conecta la base
  db.js                  Configura Sequelize y el archivo SQLite
  config.js              Puerto y origen del frontend
  models/                Un modelo Sequelize por tabla
    Producto.js
    Usuario.js
    Ticket.js
    TicketProducto.js
    associations.js      Relaciones entre las tablas
  services/              Operaciones de consulta y escritura
    product.service.js
    user.service.js
```

Seguí el recorrido: `src/index.js` monta una ruta; la ruta llama un servicio; el servicio usa un modelo; el modelo representa una tabla de `dietetica.db`.

Paso 3. Conectar Sequelize sin reemplazar los datos

`src/db.js` localiza `dietetica.db` que está en la carpeta raíz del backend y configura Sequelize con `dialect: "sqlite"`.

Al arrancar, `src/index.js` ejecuta `sequelize.authenticate()` para comprobar la conexión. No importa ni crea datos iniciales: lee los que ya están cargados.

> **Importante:** no ejecutar `sequelize.sync()`, `sync({ force: true })` ni `sync({ alter: true })` sobre esta base. El archivo ya tiene datos y el trigger de stock; evitar sync mantiene el esquema existente sin recrearlo.

Paso 4. Relacionar modelos y tablas

Un modelo describe columnas; `tableName` le dice a Sequelize el nombre exacto de la tabla que ya existe. Por ejemplo, el modelo `Producto` apunta a la tabla SQLite `Producto` y describe su título, descripción, imagen, inventario, precio y peso.

Modelo Sequelize	Tabla SQLite	Relación
Usuario	Usuario	Un usuario puede tener varios tickets.
Producto	Producto	Un producto puede aparecer en detalles de ticket.
Ticket	Ticket	Cada ticket pertenece a un usuario.
TicketProducto	Ticket_Producto	Une un ticket y un producto, y registra la cantidad.

`src/models/associations.js` expresa esas relaciones para Sequelize. `timestamps: false` es importante porque las tablas actuales no tienen columnas createdAt y updatedAt.

Paso 5. Probar que Express consulta la base

Con `npm run dev` todavía en ejecución, abrí estas direcciones:

- `http://localhost:3000/api/health` — comprueba que Express está activo.
- `http://localhost:3000/api/productos` — devuelve los diez productos.
- `http://localhost:3000/api/productos/1` — busca un producto por ID.
- `http://localhost:3000/api/usuarios` — devuelve usuarios sin contraseñas.

Las rutas siguen el estilo CRUD del ejemplo: productos tiene GET (lista y por ID), POST (crear), PUT (actualizar) y DELETE (eliminar). La lista de usuarios es de solo lectura.

> Si abrís `/api/productos` y aparecen registros JSON, la conexión completa Node → Sequelize → SQLite está funcionando.

Paso 6. Mostrar productos en React con useEffect

En el componente de catálogo del frontend, solicitá los datos cuando React monte el componente:

```
import { useEffect, useState } from "react";

function Catalogo() {
  const [productos, setProductos] = useState([]);
```

```
  const [error, setError] = useState("");

  useEffect(() => {
    const controller = new AbortController();

    async function cargarProductos() {
      try {
        const respuesta = await fetch(
          "http://localhost:3000/api/productos",
          { signal: controller.signal }
        );
        if (!respuesta.ok) throw new Error(`HTTP ${respuesta.status}`);
        setProductos(await respuesta.json());
      } catch (err) {
        if (err.name !== "AbortError") {
          setError("No se pudieron cargar los productos.");
          console.error(err);
        }
      }
    }

    cargarProductos();
    return () => controller.abort();
  }, []);

  if (error) return <p role="alert">{error}</p>;
  return (
    <ul>
      {productos.map((producto) => (
        <li key={producto.id_producto}>
          {producto.nombre_producto} — ${producto.precio}
        </li>
      ))}
    </ul>
  );
}

export default Catalogo;
```

Paso 7. Correr frontend y backend juntos

1. Terminal 1, carpeta Backend: `npm run dev`.
2. Terminal 2, carpeta Frontend: `npm run dev`.
3. Comprobá que la terminal del frontend indique el puerto que usa Vite (normalmente 5173).
4. Si Vite usa otro puerto, creá `.env` en Backend copiando `.env.example` y cambiá `FRONTEND_ORIGIN` por el origen exacto del frontend.
5. Abrí el frontend: el catálogo debe consultar `http://localhost:3000/api/productos` y mostrar la respuesta.

El middleware CORS permite las solicitudes del origen local configurado. CORS autoriza al navegador; no conecta la base. Esa conexión la hace Sequelize dentro de Node.

Cuando quieras agregar otra funcionalidad

1. Describí la tabla con un modelo Sequelize y su nombre exacto.
2. Escribí la consulta y validación en un servicio.
3. Enlazá el servicio a una ruta Express.
4. Montá esa ruta en `src/index.js`.
5. Consumí el endpoint desde React con `fetch`, normalmente dentro de `useEffect` para una carga inicial.
6. Probá primero la URL del backend y luego probá la pantalla React.

Datos y seguridad

- Los diez productos tienen stock 0 porque el JSON original no proporcionaba inventario.
- El trigger `trg_descuento_stock` sigue en SQLite. Valida el stock y lo descuenta al insertar una línea de ticket.
- La API omite el campo `password` al listar usuarios. Las contraseñas actuales son de demostración y están en texto plano; no deben usarse para login real.
- Los datos de productos enviados a POST/PUT se validan en el servidor antes de guardarse.

La implementación y los pasos detallados también están en README.md dentro de la carpeta Backend.